

RESOLUCIÓN DE EJERCICIOS SEMANA 12

Estudiante: Cuba Villanueva Javier Amilcar

TEMA 1. ESTRATEGIAS DE BACKUP: COMPLETO, DIFERENCIAL Y TRANSACCIONAL

1. Enunciado del Ejercicio

La Oficina de Tecnologías de Información de la universidad requiere implementar una política de respaldo completo diario para la base de datos GradosTitulos, la cual almacena información crítica de estudiantes, trámites, evaluaciones, resoluciones y diplomas.

El objetivo es generar un respaldo completo de la base de datos y registrar la fecha de ejecución del proceso.

2. Script de la solución en SQL

```
USE master;
GO

BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Users\Usuario\GradosTitulos_Completo.bak'
WITH
FORMAT,
INIT,
NAME = 'Backup Completo GradosTitulos',
DESCRIPTION = 'Respaldo completo institucional',
STATS = 10;

PRINT 'Respaldo realizado el: ' + CONVERT(VARCHAR, GETDATE(), 120);
```

3. Justificación Técnica de la solución aplicada

Se eligió un respaldo completo porque este tipo de backup almacena una copia íntegra de toda la base de datos GradosTitulos, incluyendo las tablas de los esquemas Academico, Tramite, Evaluacion y Documento.

La solución permite restaurar la base de datos completa en caso de fallas graves, corrupción de archivos o pérdida total del servidor.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se utiliza la base master para ejecutar tareas administrativas de respaldo.
- Se asigna un nombre descriptivo al backup mediante la opción NAME.
- Se agrega una descripción para facilitar auditorías y mantenimiento.
- Se activa la compresión para optimizar espacio en disco.
- Se utiliza CHECKSUM para detectar posibles errores durante el respaldo.
- Se registra la fecha y hora de ejecución mediante PRINT y GETDATE().
- El archivo de respaldo se almacena en una carpeta dedicada (C:\Backup) para mantener organizada la estrategia de recuperación.

Proyecto 2: Implementación de Backups Diferenciales para Trámites Académicos

1. Enunciado del Ejercicio

La universidad necesita reducir los tiempos de respaldo debido al crecimiento de las tablas de los esquemas Tramite y Documento.

Se requiere una estrategia que combine un respaldo completo semanal y respaldos diferenciales diarios.

2. Script de la solución en SQL

```
--1.2  
  
BACKUP DATABASE GradosTitulos  
TO DISK='C:\Users\Usuario\GradosTitulos_Completo.bak'  
WITH INIT;  
GO
```

3. Justificación Técnica de la solución aplicada

El respaldo diferencial solo guarda los cambios realizados desde el último respaldo completo, lo que reduce considerablemente el tiempo de ejecución y el espacio utilizado en disco.

En la base GradosTitulos, las tablas de trámites, observaciones, resoluciones y documentos pueden crecer rápidamente debido al procesamiento continuo de expedientes académicos. Por ello, un respaldo diferencial diario resulta más eficiente que realizar respaldos completos todos los días.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se mantiene un respaldo completo como base de recuperación.
- Los respaldos diferenciales se ejecutan diariamente para minimizar el tiempo de respaldo.
- Se utiliza compresión para optimizar almacenamiento.
- Se aplica CHECKSUM para validar la integridad del respaldo.
- La estrategia reduce la carga sobre el servidor SQL Server durante horarios de operación académica.
- Los nombres de archivos identifican claramente el tipo de respaldo realizado.

Proyecto 3: Estrategia Integral Full + Differential + Log Backup

1. Enunciado del Ejercicio

La base de datos GradosTitulos opera las 24 horas y registra continuamente solicitudes de grados y títulos.

Se requiere implementar una estrategia de respaldo que incluya:

- Respaldo completo semanal.
- Respaldo diferencial diario.
- Respaldo del log de transacciones cada 30 minutos.

2. Script de la solución en SQL

```

25
26  BACKUP DATABASE GradosTitulos
27  TO DISK='C:\Users\Usuario\Semanal.bak'
28  WITH INIT;
29  GO
30
31  BACKUP DATABASE GradosTitulos
32  TO DISK='C:\Users\Usuario\Diario.bak'
33  WITH DIFFERENTIAL;
34  GO
35
36  ALTER DATABASE GradosTitulos
37  SET RECOVERY FULL;
38  GO
39
40  BACKUP LOG GradosTitulos
41  TO DISK='C:\Users\Usuario\Log30Min.trn'
42  WITH INIT;
43  GO

```

3. Justificación Técnica de la solución aplicada

El modelo de recuperación FULL permite conservar todas las transacciones registradas en el log, haciendo posible recuperar la base de datos hasta un punto específico en el tiempo.

Esta estrategia es adecuada para GradosTitulos porque la pérdida de información sobre trámites, evaluaciones o resoluciones puede generar problemas administrativos y legales.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se configura explícitamente el modelo de recuperación FULL.
- Los respaldos del log se realizan con frecuencia para reducir la pérdida potencial de información.
- Se utiliza compresión para ahorrar espacio.
- CHECKSUM ayuda a detectar corrupción durante el respaldo.
- La estrategia sigue el principio 3-2-1 de protección de datos cuando los archivos se copian posteriormente a otros medios de almacenamiento.

Proyecto 4: Estrategia de Respaldo para Alta Disponibilidad

1. Enunciado del Ejercicio

Se requiere definir una estrategia de recuperación ante desastres para la plataforma institucional de grados y títulos, garantizando una pérdida máxima de información de 15 minutos.

2. Script de la solución en SQL

```
--1.4
|
|
| ALTER DATABASE GradosTitulos
| SET RECOVERY FULL;
| GO
|
| BACKUP DATABASE GradosTitulos
| TO DISK='C:\Users\Usuario\Completo.bak'
| WITH INIT;
|
| BACKUP DATABASE GradosTitulos
| TO DISK='C:\Users\Usuario\Diferencial.bak'
| WITH DIFFERENTIAL;
|
| BACKUP LOG GradosTitulos
| TO DISK='C:\Users\Usuario\Log.trn';
| GO
```

3. Justificación Técnica de la solución aplicada

La frecuencia de respaldo del log de transacciones cada 15 minutos permite que, en caso de falla del servidor, la pérdida máxima de información sea de aproximadamente 15 minutos (RPO = 15 minutos).

Esto es especialmente importante para la base GradosTitulos, donde continuamente se registran trámites, pagos, evaluaciones y resoluciones académicas.

La estrategia mejora la disponibilidad y reduce el impacto operativo ante fallas de hardware, errores humanos o corrupción de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se utiliza el modelo FULL para permitir respaldos del log.
- Se establece una frecuencia de 15 minutos para cumplir el objetivo de recuperación.
- Los respaldos deben programarse mediante SQL Server Agent para ejecutarse automáticamente.
- Se recomienda almacenar los archivos de respaldo en un disco distinto al de la base de datos.
- Se mantiene una cadena continua de respaldos del log para garantizar una restauración correcta.

Proyecto 5: Implementación de una Estrategia Corporativa de Respaldo

1. Enunciado del Ejercicio

La universidad requiere una estrategia avanzada de respaldo que incluya:

- Respaldo completo cada domingo.
- Respaldo diferencial diario.
- Respaldo del log cada 15 minutos.

- Compresión de respaldos.
- Verificación automática de integridad.

2. Script de la solución en SQL

```

64
65  ✓ BACKUP DATABASE GradosTitulos
66    TO DISK='C:\Backup\CompletoDomingo.bak'
67    WITH
68    COMPRESSION,
69    CHECKSUM,
70    INIT;
71    GO
72
73  ✓ BACKUP DATABASE GradosTitulos
74    TO DISK='C:\Backup\Diferencial.bak'
75    WITH
76    DIFFERENTIAL,
77    COMPRESSION,
78    CHECKSUM;
79    GO
80
81  ✓ BACKUP LOG GradosTitulos
82    TO DISK='C:\Backup\Log15Min.trn'
83    WITH
84    COMPRESSION,
85    CHECKSUM;
86    GO
87
88  ✓ RESTORE VERIFYONLY
89    FROM DISK='C:\Backup\CompletoDomingo.bak';
90    GO

```

3. Justificación Técnica de la solución aplicada

La solución implementa una estrategia corporativa completa que combina protección de datos, optimización de almacenamiento y verificación de integridad.

RESTORE VERIFYONLY permite comprobar que el archivo de respaldo puede ser leído correctamente y que no presenta errores de estructura, aumentando la confiabilidad del proceso de recuperación.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se comprime el respaldo para reducir costos de almacenamiento.
- Se utiliza CHECKSUM para detectar corrupción.
- Se verifica automáticamente la integridad del archivo de respaldo.
- Se separan respaldos completos, diferenciales y de log en archivos identificables.
- La estrategia es escalable y adecuada para ambientes institucionales de producción.

Proyecto 6: Estrategia de Respaldo para Recuperación ante Desastres

1. Enunciado del Ejercicio

La Oficina de Tecnologías de Información requiere una estrategia de Disaster Recovery que permita reconstruir completamente la base de datos GradosTitulos en un servidor alternativo ante la pérdida total del servidor principal.

2. Script de la solución en SQL

```
92  --1.6
93
94  ✓ BACKUP DATABASE GradosTitulos
95    TO DISK='C:\Backup\Completo.bak';
96    GO
97
98  ✓ BACKUP DATABASE GradosTitulos
99    TO DISK='C:\Backup\Diferencial.bak'
100    WITH DIFFERENTIAL;
101    GO
102
103    BACKUP LOG GradosTitulos
104    TO DISK='C:\Backup\Log.trn';
105    GO
```

3. Justificación Técnica de la solución aplicada

La estrategia permite reconstruir completamente la base de datos en un servidor alternativo utilizando la secuencia:

- Respaldo completo.
- Respaldo diferencial más reciente.
- Todos los respaldos del log generados posteriormente.

De esta forma se puede recuperar prácticamente toda la información almacenada en las tablas de GradosTitulos, incluyendo expedientes, pagos, evaluaciones, resoluciones y diplomas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Se utiliza el modelo de recuperación FULL para mantener la cadena de logs.
- Se combinan tres tipos de respaldo para maximizar la capacidad de recuperación.
- Se verifica la integridad del respaldo antes de considerarlo válido.
- Se recomienda copiar los archivos de respaldo a un servidor alternativo o almacenamiento externo.
- La estrategia está alineada con planes de continuidad operativa y recuperación ante desastres (Disaster Recovery).

TEMA 2. RESTAURACIÓN DE BASES DE DATOS EN DISTINTOS ESCENARIOS

1. Enunciado del Ejercicio

Un administrador eliminó accidentalmente registros relacionados con expedientes de titulación de la base de datos **GradosTitulos**.

Se requiere restaurar la base de datos utilizando el respaldo más reciente disponible para recuperar la información perdida.

2. Script de la solución en SQL

```
USE master;
GO

-- Restaurar la base de datos desde el respaldo completo más reciente

RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backup\GradosTitulos_Completo.bak'
WITH
REPLACE,
RECOVERY,
STATS = 10;
GO
```

3. Justificación Técnica de la solución aplicada

Se utiliza el respaldo completo más reciente porque contiene una copia íntegra de toda la base de datos **GradosTitulos**, incluyendo las tablas de los esquemas **Academico**, **Tramite**, **Evaluacion**, **Documento** y **Catalogo**.

La opción **REPLACE** permite sobrescribir la base de datos existente cuando es necesario restaurarla sobre la misma instancia.

La opción **RECOVERY** deja la base completamente operativa una vez finalizado el proceso de restauración.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Ejecutar la restauración desde la base **master**.
- Mantener respaldos completos actualizados.
- Utilizar **STATS** para monitorear el progreso.
- Restaurar únicamente respaldos previamente verificados.
- Realizar la restauración durante ventanas de mantenimiento para evitar afectar a los usuarios.

Proyecto 2: Restauración en un Servidor Alternativo

1. Enunciado del Ejercicio

La universidad requiere habilitar un servidor de contingencia para realizar pruebas y recuperación ante fallas.

Se debe restaurar la base de datos **GradosTitulos** en una nueva instancia de SQL Server.

2. Script de la solución en SQL

```

14 USE master;
15 GO
16
17 RESTORE DATABASE GradosTitulos
18 FROM DISK='C:\Backup\GradosTitulos_Completo.bak'
19 WITH
20 MOVE 'GradosTitulos' TO 'D:\SQLData\GradosTitulos.mdf',
21 MOVE 'GradosTitulos_log' TO 'D:\SQLData\GradosTitulos_log.ldf',
22 REPLACE,
23 RECOVERY,
24 STATS=10;
25 GO

```

3. Justificación Técnica de la solución aplicada

Cuando la restauración se realiza en otro servidor, normalmente las rutas donde se almacenan los archivos de datos son diferentes.

La cláusula **MOVE** permite indicar la nueva ubicación física de los archivos MDF y LDF, evitando errores durante la restauración.

Esta técnica es utilizada para implementar servidores de contingencia, ambientes de pruebas y recuperación ante desastres.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Utilizar discos diferentes para datos y transacciones.
- Mantener organizada la ubicación de los archivos MDF y LDF.
- Verificar previamente los nombres lógicos mediante RESTORE FILELISTONLY.
- Restaurar siempre sobre un servidor con una versión compatible de SQL Server.

Proyecto 3: Recuperación después de Corrupción del Archivo MDF

1. Enunciado del Ejercicio

El archivo principal de datos (MDF) presenta corrupción física, impidiendo acceder correctamente a la base de datos.

Se requiere restaurar completamente la base utilizando los respaldos disponibles.

2. Script de la solución en SQL

```

39 USE master;
40 GO
41
42 RESTORE DATABASE GradosTitulos
43 FROM DISK='C:\Backup\GradosTitulos_Completo.bak'
44 WITH
45 NORECOVERY,
46 REPLACE;
47
48 GO
49
50 RESTORE DATABASE GradosTitulos
51 FROM DISK='C:\Backup\GradosTitulos_Diferencial.bak'
52 WITH
53 RECOVERY;
54 GO

```

3. Justificación Técnica de la solución aplicada

Cuando el archivo MDF presenta corrupción, la forma más segura de recuperar la información consiste en restaurar la base desde un respaldo válido.

Se utiliza **NORECOVERY** para mantener la base en estado de restauración mientras se aplican respaldos adicionales.

Finalmente **RECOVERY** deja la base lista para su utilización.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Nunca intentar reparar directamente un MDF corrupto sin contar con respaldos.
- Mantener respaldos diferenciales actualizados.
- Conservar la secuencia correcta de restauración.
- Verificar la integridad del respaldo antes de utilizarlo.

Proyecto 4: Recuperación ante Desastre Total del Servidor

1. Enunciado del Ejercicio

Una falla de almacenamiento destruyó completamente el servidor donde se encontraba la base de datos **GradosTitulos**.

Se requiere reconstruir completamente la base utilizando todos los respaldos disponibles.

2. Script de la solución en SQL

```
158 USE master;
159 GO
160
161 RESTORE DATABASE GradosTitulos
162 FROM DISK='C:\Backup\GradosTitulos_Completo.bak'
163 WITH
164 NORECOVERY,
165 REPLACE;
166
167 GO
168
169 RESTORE DATABASE GradosTitulos
170 FROM DISK='C:\Backup\GradosTitulos_Diferencial.bak'
171 WITH
172 NORECOVERY;
173
174 GO
175
176 RESTORE LOG GradosTitulos
177 FROM DISK='C:\Backup\GradosTitulos_Log.trn'
178 WITH
179 RECOVERY;
180 GO
```

3. Justificación Técnica de la solución aplicada

La estrategia utiliza la cadena completa de recuperación:

- Respaldo completo.
- Respaldo diferencial.

- Respaldo del log de transacciones.

De esta manera se recupera prácticamente toda la información registrada hasta el momento de la última copia del log de transacciones, minimizando la pérdida de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Mantener el modelo de recuperación **FULL**.
- No interrumpir la secuencia de respaldos del log.
- Restaurar siempre en el orden correcto.
- Almacenar copias de respaldo en medios externos al servidor principal.

Proyecto 5: Recuperación Completa después de Corrupción del Archivo MDF

1. Enunciado del Ejercicio

Se detectó corrupción física en el archivo principal de datos de la base **GradosTitulos**, impidiendo el acceso a las tablas relacionadas con expedientes de titulación.

Se requiere recuperar completamente la base utilizando la secuencia de respaldos disponible.

2. Script de la solución en SQL

```
183
184     USE master;
185     GO
186
187     RESTORE DATABASE GradosTitulos
188     FROM DISK='C:\Backup\GradosTitulos_Completo.bak'
189     WITH
190     NORECOVERY,
191     REPLACE;
192
193     GO
194
195     RESTORE DATABASE GradosTitulos
196     FROM DISK='C:\Backup\GradosTitulos_Diferencial.bak'
197     WITH
198     NORECOVERY;
199
200     GO
201
202     RESTORE LOG GradosTitulos
203     FROM DISK='C:\Backup\GradosTitulos_Log.trn'
204     WITH
205     RECOVERY;
206     GO
```

3. Justificación Técnica de la solución aplicada

La corrupción del archivo MDF imposibilita acceder a la información almacenada en las tablas de la base de datos.

La restauración secuencial permite reconstruir completamente la base y recuperar la información correspondiente a estudiantes, matrículas, trámites, evaluaciones, resoluciones y diplomas.

La aplicación del respaldo del log garantiza que las transacciones registradas después del respaldo diferencial también sean recuperadas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Realizar respaldos completos de forma periódica.
- Complementar con respaldos diferenciales.
- Generar respaldos frecuentes del log de transacciones.
- Validar periódicamente la posibilidad de restaurar los respaldos mediante pruebas de recuperación.

Proyecto 6: Restauración de la Base de Datos en un Servidor de Contingencia

1. Enunciado del Ejercicio

La universidad requiere habilitar un centro alternativo de recuperación para continuar los procesos de grados y títulos en caso de que el servidor principal deje de funcionar.

Se debe restaurar la base de datos en una instancia secundaria de SQL Server.

2. Script de la solución en SQL

```
3  --2.6
3  USE master;
3  GO
1
2  RESTORE DATABASE GradosTitulos
3  FROM DISK='C:\Backup\GradosTitulos_Completo.bak'
4  WITH
5  MOVE 'GradosTitulos' TO 'E:\Contingencia\GradosTitulos.mdf',
5  MOVE 'GradosTitulos_log' TO 'E:\Contingencia\GradosTitulos_log.ldf',
7  REPLACE,
3  RECOVERY,
3  STATS=10;
3  GO
```

3. Justificación Técnica de la solución aplicada

La restauración en un servidor de contingencia garantiza la continuidad de las operaciones institucionales cuando el servidor principal presenta una falla crítica.

La cláusula **MOVE** permite adaptar la restauración a la estructura de almacenamiento del nuevo servidor, mientras que **RECOVERY** deja la base lista para recibir conexiones de los usuarios.

Esta estrategia es fundamental dentro de un plan de continuidad del negocio y recuperación ante desastres.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- Mantener un servidor alternativo preparado para contingencias.
- Almacenar los respaldos en un medio diferente al servidor principal.

- Verificar periódicamente que la restauración funcione correctamente mediante simulacros.
- Documentar el procedimiento de recuperación para reducir el tiempo de respuesta ante un desastre.
- Mantener sincronizada la estrategia de respaldos entre el servidor principal y el servidor de contingencia.

TEMA 3 - Proyecto 1: Verificación de Integridad de Backups

1. Enunciado del Ejercicio

Verifique que todos los respaldos generados para GradosTitulos sean recuperables.

2. Script de la solución en SQL

```
USE master;
GO

-- Declarar la ruta del archivo a verificar
DECLARE @RutaArchivoBackup NVARCHAR(500) = 'C:\Backups\GradosTitulos_Full.bak';

-- Ejecutar la verificación de integridad
RESTORE VERIFYONLY
FROM DISK = @RutaArchivoBackup;

IF @@ERROR = 0
    PRINT 'EXITO: El archivo de respaldo es válido y su integridad está intacta.';
ELSE
    PRINT 'ERROR: El archivo de respaldo está corrupto o es inaccesible.';
GO
```

3. Justificación Técnica de la solución aplicada

Se utiliza la instrucción RESTORE VERIFYONLY. A diferencia de un RESTORE tradicional, este comando no sobrescribe ni modifica la base de datos actual. Su única función es leer los encabezados del archivo .bak y verificar que el medio de almacenamiento sea legible y que la estructura interna del backup (páginas y sumas de comprobación) esté consistente.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Seguridad de Datos:** Realizar esta verificación asegura que los respaldos críticos no estén corruptos antes de que ocurra un desastre, garantizando los objetivos de punto de recuperación (RPO).
- **Manejo de Variables:** Centralizar la ruta del archivo en una variable (@RutaArchivoBackup) facilita la actualización y reutilización del script.

TEMA 3 - Proyecto 2: Inventario Histórico de Copias de Seguridad

1. Enunciado del Ejercicio

Construya un reporte que muestre todos los respaldos realizados durante el último mes.

2. Script de la solución en SQL

```
9 USE msdb;
9 GO
1
2 SELECT
3     bs.database_name AS BaseDeDatos,
4     bs.backup_start_date AS FechaInicio,
5     bs.backup_finish_date AS FechaFin,
6     CASE bs.type
7         WHEN 'D' THEN 'Completo (Full)'
8         WHEN 'I' THEN 'Diferencial'
9         WHEN 'L' THEN 'Log de Transacciones'
10        ELSE 'Otro'
11    END AS TipoBackup,
12     CAST(bs.backup_size / 1048576.0 AS DECIMAL(10,2)) AS Tamano_MB,
13     bmf.physical_device_name AS RutaFisica
14 FROM msdb.dbo.backupset AS bs
15 INNER JOIN msdb.dbo.backupmediafamily AS bmf
16     ON bs.media_set_id = bmf.media_set_id
17 WHERE bs.database_name = 'GradosTitulos'
18     -- Filtro para obtener solo los registros del último mes
19     AND bs.backup_finish_date >= DATEADD(MONTH, -1, GETDATE())
20 ORDER BY bs.backup_finish_date DESC;
21 GO
```

3. Justificación Técnica de la solución aplicada

El historial de respaldos de SQL Server se almacena en la base de datos del sistema msdb. Se realiza un INNER JOIN entre las tablas backupset (que contiene metadatos como fecha, tipo y tamaño) y backupmediafamily (que contiene la ruta del archivo). La función DATEADD(MONTH, -1, GETDATE()) calcula dinámicamente la fecha de hace exactamente un mes.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Formateo Amigable:** Se convierte el tamaño del archivo de bytes a Megabytes (dividiendo entre 1048576.0) para que sea humanamente legible.
- **Ordenamiento Lógico:** Se ordena de manera descendente (DESC) por fecha de finalización, priorizando en la lectura los respaldos más recientes.

TEMA 3 - Proyecto 3: Detección de Respaldos Obsoletos

1. Enunciado del Ejercicio

Identifique copias de seguridad que excedan la política institucional de retención.

2. Script de la solución en SQL

```

263 USE msdb;
264 GO
265 
266 -- Definir política de retención (ejemplo: 30 días)
267 DECLARE @DiasRetencion INT = 30;
268 DECLARE @FechaLimite DATETIME = DATEADD(DAY, -@DiasRetencion, GETDATE());
269 
270 SELECT
271     bs.database_name AS BaseDeDatos,
272     bs.backup_finish_date AS FechaGeneracion,
273     DATEDIFF(DAY, bs.backup_finish_date, GETDATE()) AS DiasAntiguedad,
274     bmf.physical_device_name AS ArchivoObsoleto
275 FROM msdb.dbo.backupset AS bs
276 INNER JOIN msdb.dbo.backupmediafamily AS bmf
277     ON bs.media_set_id = bmf.media_set_id
278 WHERE bs.database_name = 'GradosTitulos'
279     AND bs.backup_finish_date < @FechaLimite
280 ORDER BY bs.backup_finish_date ASC;
281 GO

```

3. Justificación Técnica de la solución aplicada

Se define una variable @DiasRetencion que representa la política de la empresa. Se utiliza la función DATEDIFF para calcular exactamente cuántos días han pasado desde la creación del backup hasta hoy, filtrando con la cláusula WHERE aquellos que superan la fecha límite calculada.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Parametrización de Reglas de Negocio:** Al encapsular la cantidad de días en la variable @DiasRetencion, el script se adapta inmediatamente si la institución decide cambiar su política de 30 a 60 días en el futuro.
- **Gestión de Almacenamiento:** Identificar proactivamente estos archivos es el primer paso antes de programar rutinas de borrado, evitando que los discos del servidor colapsen por falta de espacio.

TEMA 3 - Proyecto 4: Sistema Automatizado de Verificación

1. Enunciado del Ejercicio

Diseña una solución que valide diariamente la integridad de todos los respaldos de GradosTitulos.

2. Script de la solución en SQL

```

13 USE GradosTitulos;
14 GO
15
16 CREATE OR ALTER PROCEDURE dbo.SP_VerificarUltimoRespaldo
17 AS
18 BEGIN
19     SET NOCOUNT ON;
20     DECLARE @UltimoBackup NVARCHAR(500);
21     DECLARE @ComandoSQL NVARCHAR(MAX);
22
23     -- 1. Obtener la ruta física del respaldo más reciente
24     SELECT TOP 1 @UltimoBackup = bmf.physical_device_name
25     FROM msdb.dbo.backupset bs
26     INNER JOIN msdb.dbo.backupmediafamily bmf
27     ON bs.media_set_id = bmf.media_set_id
28     WHERE bs.database_name = 'GradosTitulos'
29     AND bs.type = 'D' -- Solo respaldos completos
30     ORDER BY bs.backup_finish_date DESC;
31
32     -- 2. Ejecutar la validación si existe el archivo
33     IF @UltimoBackup IS NOT NULL
34     BEGIN
35         BEGIN TRY
36             SET @ComandoSQL = 'RESTORE VERIFYONLY FROM DISK = ''' + @UltimoBackup + ''';
37             EXEC sp_executesql @ComandoSQL;
38             PRINT 'Verificación exitosa para el archivo: ' + @UltimoBackup;
39         END TRY
40         BEGIN CATCH
41             PRINT 'ERROR DE INTEGRIDAD en el archivo: ' + @UltimoBackup;
42             PRINT ERROR_MESSAGE();
43         END CATCH
44     END
45     ELSE
46     BEGIN
47         PRINT 'No se encontraron respaldos recientes para verificar.';
48     END
49 END;
50 GO

```

3. Justificación Técnica de la solución aplicada

La "solución" se diseña mediante un Procedimiento Almacenado (Stored Procedure) que encapsula la lógica. El script consulta msdb usando TOP 1 y ORDER BY ... DESC para localizar de forma dinámica el último backup completo generado. Luego, construye y ejecuta la verificación mediante SQL dinámico (sp_executesql).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Manejo de Errores Estructurado:** El uso del bloque TRY...CATCH asegura que si el archivo fue eliminado físicamente o está corrupto, el procedimiento no se rompa de manera abrupta, sino que capture el error y lo notifique de manera limpia.
- **Encapsulamiento Lógico:** Convertir el script en un PROCEDURE permite que sea llamado desde cualquier herramienta externa o trabajo programado sin reescribir código.

TEMA 3 - Proyecto 5: Sistema Automatizado de Validación de Backups

1. Enunciado del Ejercicio

Diseñe un mecanismo que valide diariamente todos los respaldos de la base GradosTitulos.

2. Script de la solución en SQL

```
12 USE msdb;
13 GO
14
15 -- Dado que el Proyecto 4 construyó la lógica (SP), el Proyecto 5 construye
16 -- el MECANISMO automatizado (SQL Server Agent Job) para ejecutarlo diariamente.
17
18 EXEC dbo.sp_add_job
19     @job_name = N'Job_ValidacionDiaria_GradosTitulos',
20     @enabled = 1,
21     @description = N'Ejecuta la validación de integridad del último backup diariamente.';
22 GO
23
24 EXEC dbo.sp_add_jobstep
25     @job_name = N'Job_ValidacionDiaria_GradosTitulos',
26     @step_name = N'Paso 1: Ejecutar SP de Verificación',
27     @subsystem = N'TSQL',
28     @command = N'EXEC GradosTitulos.dbo.SP_VerificarUltimoRespaldo;',
29     @database_name = N'master';
30 GO
31
32 -- Programar el Job para que se ejecute todos los días a las 02:00 AM
33 EXEC dbo.sp_add_schedule
34     @schedule_name = N'Horario_Madrugada',
35     @freq_type = 4, -- 4 = Diariamente
36     @freq_interval = 1,
37     @active_start_time = 020000; -- Formato HHMMSS
38 GO
39
40 -- Adjuntar el horario al Job
41 EXEC dbo.sp_attach_schedule
42     @job_name = N'Job_ValidacionDiaria_GradosTitulos',
43     @schedule_name = N'Horario_Madrugada';
44 GO
45
46 EXEC dbo.sp_add_jobserver
47     @job_name = N'Job_ValidacionDiaria_GradosTitulos';
48 GO
```

3. Justificación Técnica de la solución aplicada

Mientras que el Ejercicio 4 requería la *solución* (el código de validación), el Ejercicio 5 requiere un *mecanismo automatizado diario*. La herramienta nativa en SQL Server para esto es el **SQL Server Agent**. Se utilizan los procedimientos del sistema de msdb (sp_add_job, sp_add_jobstep, sp_add_schedule) para crear una tarea programada que dispare el procedimiento almacenado creado anteriormente todas las madrugadas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Automatización Desatendida:** Extraer la carga operativa del DBA. El sistema ahora verifica la integridad por sí solo en ventanas de bajo tráfico (2:00 AM).
- **Separación de Responsabilidades:** La lógica de negocio reside en el Stored Procedure, mientras que la lógica de calendarización reside en el Agent Job. Esto permite modificar los horarios sin tener que alterar el código de la base de datos.

TEMA 3 - Proyecto 6: Auditoría Integral del Historial de Copias de Seguridad

1. Enunciado del Ejercicio

La Dirección de TI solicita un reporte detallado de los respaldos ejecutados sobre GradosTitulos durante los últimos 90 días.

2. Script de la solución en SQL

```
USE msdb;
Cambio: no guardado

SELECT
    bs.database_name AS BaseDeDatos,
    bs.user_name AS UsuarioEjecutor,
    bs.backup_start_date AS InicioOperacion,
    bs.backup_finish_date AS FinOperacion,
    -- Calcular la duración en minutos
    DATEDIFF(MINUTE, bs.backup_start_date, bs.backup_finish_date) AS Duracion_Minutos,
    CASE bs.type
        WHEN 'D' THEN 'FULL'
        WHEN 'I' THEN 'DIFERENCIAL'
        WHEN 'L' THEN 'LOG'
    END AS Tipo,
    -- Tamaño con formato en Gigabytes para auditorías gerenciales
    CAST(bs.backup_size / 1073741824.0 AS DECIMAL(10,3)) AS Tamano_GB,
    bs.server_name AS ServidorOrigen,
    bmf.physical_device_name AS UbicacionArchivo
FROM msdb.dbo.backupset AS bs
INNER JOIN msdb.dbo.backupmediafamily AS bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'GradosTitulos'
    AND bs.backup_start_date >= DATEADD(DAY, -90, GETDATE())
ORDER BY bs.backup_start_date DESC;
GO
```

3. Justificación Técnica de la solución aplicada

Dado que es una auditoría "integral" exigida por la Dirección de TI, el reporte debe contener métricas avanzadas. Se agregó la columna user_name para rastrear qué usuario ejecutó el comando. Además, se utilizó DATEDIFF en minutos para analizar el rendimiento del proceso (cuánto tarda en hacerse el backup) y se formateó el tamaño a Gigabytes (dividiendo por 1,073,741,824) dado que el rango de 90 días exige un análisis de crecimiento de los datos a un nivel más macro.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Orientación a Auditoría:** El cruce de información (usuario, tiempo de duración y máquina de origen) cumple con los estándares de seguridad perimetral para identificar procesos o accesos no autorizados al sistema de respaldos.
- **Análisis de Rendimiento (Capacity Planning):** Exponer la duración en minutos y el tamaño en GB permite a la Dirección de TI prever cuándo se requerirá comprar más almacenamiento en disco o si el proceso de backup está ralentizando excesivamente el servidor.

TEMA 4 - Proyecto 1: Respaldo Manual de la Base GradosTitulos

1. Enunciado del Ejercicio

Realice un respaldo completo utilizando únicamente instrucciones T-SQL.

2. Script de la solución en SQL

```
USE master;
GO

-- Crear un respaldo completo (Full Backup) hacia un dispositivo físico
BACKUP DATABASE GradosTitulos
TO DISK = 'C:\Backups\GradosTitulos_Manual.bak'
WITH FORMAT,
INIT,
NAME = 'Respaldo Completo Manual - GradosTitulos',
STATS = 10; -- Muestra el progreso cada 10%
GO
```

3. Justificación Técnica de la solución aplicada

Se utiliza la instrucción nativa BACKUP DATABASE. Se especifican las cláusulas FORMAT e INIT para asegurar que se cree un nuevo encabezado de medios y se sobrescriba cualquier archivo anterior con el mismo nombre, evitando que el archivo .bak crezca indefinidamente al anexar respaldos uno tras otro.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Monitoreo de Ejecución:** El uso de STATS = 10 es fundamental para bases de datos grandes, ya que proporciona retroalimentación en la consola sobre el progreso de la operación, evitando que el administrador piense que el proceso se ha colgado.
- **Etiquetado Interno:** Incluir un NAME descriptivo facilita identificar el propósito del backup si este archivo se analiza en el futuro utilizando instrucciones como RESTORE HEADERONLY.

TEMA 4 - Proyecto 2: Restauración Completa desde Archivo BAK

1. Enunciado del Ejercicio Recupere la base de datos utilizando un archivo de respaldo existente.

2. Script de la solución en SQL

```
USE master;
GO

-- 1. Pasar a modo usuario único para desconectar usuarios activos
ALTER DATABASE GradosTitulos
SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
GO

-- 2. Ejecutar la restauración forzando el reemplazo
RESTORE DATABASE GradosTitulos
FROM DISK = 'C:\Backups\GradosTitulos_Manual.bak'
WITH REPLACE, RECOVERY, STATS = 10;
GO

-- 3. Devolver la base de datos a producción
ALTER DATABASE GradosTitulos
SET MULTI_USER;
GO
```

3. Justificación Técnica de la solución aplicada

Para restaurar una base de datos que está en uso, SQL Server exige que el motor tenga acceso exclusivo. La instrucción ALTER DATABASE ... SET SINGLE_USER WITH ROLLBACK IMMEDIATE interrumpe todas las transacciones abiertas. Luego, el parámetro WITH REPLACE le indica al motor que ignore las validaciones de seguridad de cola de registros y sobrescriba la base de datos.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Prevención de Bloqueos:** El ROLLBACK IMMEDIATE es una práctica crítica de DBA para evitar que un proceso de restauración falle debido a sesiones "huérfanas" conectadas a la base de datos.
- **Disponibilidad Inmediata:** La cláusula RECOVERY indica a SQL Server que no se aplicarán más archivos de registro (como backups diferenciales), dejando la base de datos lista y operativa de inmediato tras finalizar.

TEMA 4 - Proyecto 3: Automatización de Backups y Restores

1. Enunciado del Ejercicio Diseñe procedimientos almacenados que automaticen respaldos y restauraciones.

2. Script de la solución en SQL

```
USE GradosTitulos;
GO

-- Procedimiento para Automatizar Backups
CREATE OR ALTER PROCEDURE dbo.SP_AutoBackup
    @RutaDestino NVARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @ComandoSQL NVARCHAR(MAX);
    SET @ComandoSQL = 'BACKUP DATABASE GradosTitulos TO DISK = ''' + @RutaDestino + ''' WITH FORMAT, INIT, STATS=10;';
    EXEC sp_executesql @ComandoSQL;
END;
GO

-- Procedimiento para Automatizar Restores
CREATE OR ALTER PROCEDURE dbo.SP_AutoRestore
    @RutaOrigen NVARCHAR(255)
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @ComandoSQL NVARCHAR(MAX);
    -- SQL Dinámico para asegurar el cambio de contexto y restauración
    SET @ComandoSQL = '
        ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
        RESTORE DATABASE GradosTitulos FROM DISK = ''' + @RutaOrigen + ''' WITH REPLACE, RECOVERY;
        ALTER DATABASE GradosTitulos SET MULTI_USER;
    ';
    EXEC sp_executesql @ComandoSQL;
END;
GO
```

3. Justificación Técnica de la solución aplicada Se agrupan las sentencias T-SQL de los proyectos anteriores dentro de Procedimientos Almacenados (SPs). Se utiliza sp_executesql para ejecutar cadenas de texto dinámicas construidas a partir de las variables de ruta (@RutaDestino y @RutaOrigen).

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Abstracción y Seguridad:** Al utilizar SPs, un administrador puede otorgar permisos de ejecución (GRANT EXECUTE) a ciertos usuarios de TI para hacer backups sin necesidad de darles privilegios de sysadmin.
- **Reusabilidad del Código:** Evita reescribir bloques largos de código cada vez que se necesita una tarea de mantenimiento repetitiva.

TEMA 4 - Proyecto 4: Framework Institucional de Recuperación

1. Enunciado del Ejercicio

Construya un conjunto de scripts reutilizables para respaldo y restauración de todas las bases institucionales.

2. Script de la solución en SQL

```

50 USE master;
51 GO
52
53 CREATE OR ALTER PROCEDURE dbo.SP_BackupTodasLasBases
54     @DirectorioBase NVARCHAR(255) = 'C:\Backups\'
55 AS
56 BEGIN
57     SET NOCOUNT ON;
58     DECLARE @NombreBD NVARCHAR(128);
59     DECLARE @ComandoSQL NVARCHAR(MAX);
60     DECLARE @RutaFinal NVARCHAR(500);
61
62     -- Cursor para iterar sobre todas las bases de datos del servidor
63     -- excluyendo las bases de datos del sistema y tempdb
64     DECLARE CursorBases CURSOR FOR
65     SELECT name
66     FROM sys.databases
67     WHERE name NOT IN ('master', 'tempdb', 'model', 'msdb')
68     AND state_desc = 'ONLINE'; -- Solo BDs activas
69
70     OPEN CursorBases;
71     FETCH NEXT FROM CursorBases INTO @NombreBD;
72
73     WHILE @@FETCH_STATUS = 0
74     BEGIN
75         SET @RutaFinal = @DirectorioBase + @NombreBD + '_Institucional.bak';
76
77         SET @ComandoSQL = 'BACKUP DATABASE ' + QUOTENAME(@NombreBD) + '
78             TO DISK = ''' + @RutaFinal + ''' WITH FORMAT, INIT;';
79
80         PRINT 'Respaldando base de datos: ' + @NombreBD;
81         EXEC sp_executesql @ComandoSQL;
82
83         FETCH NEXT FROM CursorBases INTO @NombreBD;
84     END
85
86     CLOSE CursorBases;
87     DEALLOCATE CursorBases;
88 END;
89 GO

```

3. Justificación Técnica de la solución aplicada

El enfoque "Institucional" implica que el script debe aplicar a toda la instancia del servidor, no solo a una base de datos. Se emplea un CURSOR que lee la tabla de sistema sys.databases para identificar dinámicamente todas las bases de datos de usuario que estén operativas (ONLINE), y mediante un bucle WHILE, genera y ejecuta un respaldo individual para cada una.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Automatización a Nivel de Instancia:** Elimina el error humano. Si la institución añade una nueva base de datos al servidor mañana, este script la respaldará automáticamente sin necesidad de modificar el código.
- **Filtrado Seguro:** Se excluyen explícitamente las bases de datos del sistema (master, tempdb, etc.) ya que estas requieren estrategias de recuperación distintas.

TEMA 4 - Proyecto 5: Framework Automatizado de Respaldo Institucional

1. Enunciado del Ejercicio

Desarrolle un procedimiento almacenado reutilizable que genere respaldos completos de GradosTitulos utilizando fecha y hora en el nombre del archivo.

2. Script de la solución en SQL

```
11 USE GradosTitulos;
12 GO
13
14 CREATE OR ALTER PROCEDURE dbo.SP_RespaldoConFechaHora
15     @RutaCarpeta NVARCHAR(255) = 'C:\Backups\'
16 AS
17 BEGIN
18     SET NOCOUNT ON;
19     DECLARE @FechaActual NVARCHAR(20);
20     DECLARE @RutaArchivoCompleta NVARCHAR(500);
21     DECLARE @InstruccionBackup NVARCHAR(MAX);
22
23     -- Generar estampa de tiempo: AñoMesDia_HoraMinutoSegundo
24     SET @FechaActual = FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss');
25
26     -- Concatenar la ruta final
27     SET @RutaArchivoCompleta = @RutaCarpeta + 'GradosTitulos_' + @FechaActual + '.bak';
28
29     -- Construir el query dinámico
30     SET @InstruccionBackup = 'BACKUP DATABASE GradosTitulos
31                               TO DISK = @RutaParametro
32                               WITH FORMAT, INIT, STATS = 10;';
33
34     -- Ejecutar pasando parámetros para prevenir inyección SQL
35     EXEC sp_executesql
36         @stmt = @InstruccionBackup,
37         @params = N'@RutaParametro NVARCHAR(500)',
38         @RutaParametro = @RutaArchivoCompleta;
39
40 END;
41 GO
```

3. Justificación Técnica de la solución aplicada

Se usa la función FORMAT() aplicada a GETDATE() para extraer una estampa de tiempo estructurada. Esto se concatena con el directorio proporcionado. Se utiliza el pase de parámetros fuertemente tipados dentro de sp_executesql (@RutaParametro) en lugar de concatenar cadenas directas.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Trazabilidad Histórica (Versioning):** Al incluir fecha y hora en el nombre del archivo, se pueden mantener múltiples copias de seguridad a lo largo del día sin

sobrescribirlas, permitiendo recuperaciones a puntos muy específicos en el tiempo.

- **Seguridad de Ejecución:** El paso de parámetros formales a `sp_executesql` reduce significativamente el riesgo de vulnerabilidades estructurales en la ejecución dinámica.

TEMA 4 - Proyecto 6: Plataforma de Restauración Automatizada

1. Enunciado del Ejercicio

Construya un procedimiento que permita restaurar automáticamente GradosTitulos desde cualquier archivo BAK seleccionado.

2. Script de la solución en SQL

```
322 USE master;
323 GO
324
325 CREATE OR ALTER PROCEDURE dbo.SP_PlataformaRestauracion_GradosTitulos
326     @ArchivoBAK_Completo NVARCHAR(500)
327 AS
328 BEGIN
329     SET NOCOUNT ON;
330     BEGIN TRY
331         PRINT 'Iniciando proceso de restauración desde: ' + @ArchivoBAK_Completo;
332
333         -- 1. Asegurar la expulsión de usuarios
334         ALTER DATABASE GradosTitulos SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
335         PRINT 'Base de datos en modo exclusivo.';
336
337         -- 2. Ejecutar la restauración dinámica
338         DECLARE @SQL_Restore NVARCHAR(MAX);
339         SET @SQL_Restore = 'RESTORE DATABASE GradosTitulos
340             FROM DISK = @Archivo
341             WITH REPLACE, RECOVERY;';
342
343         EXEC sp_executesql
344             @stmt = @SQL_Restore,
345             @params = N'@Archivo NVARCHAR(500)',
346             @Archivo = @ArchivoBAK_Completo;
347
348         PRINT 'Restauración física completada exitosamente.';
349
350         -- 3. Retornar a la normalidad
351         ALTER DATABASE GradosTitulos SET MULTI_USER;
352         PRINT 'Plataforma nuevamente operativa (MULTI_USER).';
353
354     END TRY
355     BEGIN CATCH
356         -- Manejo de contingencia en caso de fallo crítico
357         PRINT 'ERROR CRÍTICO DURANTE LA RESTAURACIÓN.';
358         PRINT ERROR_MESSAGE();
359
360         -- Intentar dejar la BD utilizable si falló algo menor
361         IF DB_ID('GradosTitulos') IS NOT NULL
362         BEGIN
363             ALTER DATABASE GradosTitulos SET MULTI_USER;
364             END THROW
365         END CATCH
366 END;
367 GO
```

3. Justificación Técnica de la solución aplicada

Se diseña un SP robusto que recibe la ruta absoluta del archivo objetivo. La innovación

de este framework recae en la estructura TRY...CATCH. Engloba los tres pasos críticos (bloqueo, restauración, desbloqueo). Si el archivo no existe o está corrupto, la restauración fallará, y el flujo saltará al CATCH para notificar el error exacto.

4. Explicación de las buenas prácticas utilizadas en el proyecto

- **Resiliencia Operativa:** Si el RESTORE falla, la base de datos podría quedarse permanentemente en SINGLE_USER. El bloque CATCH incluye una instrucción de rescate que intenta forzar el MULTI_USER para no dejar el sistema inoperable por un archivo corrupto.
- **Logging en Consola:** El uso progresivo de PRINT actúa como un log que le informa al DBA exactamente en qué paso se encuentra el framework automatizado, facilitando el diagnóstico.

TEMA 5 — Planes de Mantenimiento y Políticas de Retención

Proyecto 1: Plan de Mantenimiento Semanal

1. Enunciado

Se requiere crear un Plan de Mantenimiento Semanal para la base de datos GradosTitulos. El plan debe ejecutarse automáticamente cada semana e incluir las tareas esenciales de mantenimiento: backup completo, reorganización de índices y actualización de estadísticas, garantizando el correcto funcionamiento de la base de datos.

2. Script de Solución SQL

```

571 --5.1
572 USE msdb;
573 GO
574
575 EXEC sp_add_job
576     @job_name = N'Mantenimiento_Semanal_GradosTitulos',
577     @description = N'Plan de mantenimiento semanal: backup, indices y estadísticas',
578     @owner_login_name = N'sa';
579 GO
580
581 EXEC sp_add_jobstep
582     @job_name = N'Mantenimiento_Semanal_GradosTitulos',
583     @step_name = N'Backup_Completo',
584     @step_id = 1,
585     @subsystem = N'TSQL',
586     @command = N'
587 DECLARE @ruta NVARCHAR(300);
588 SET @ruta = N'C:\SQLBackup\GradosTitulos\GradosTitulos_'
589     + CONVERT(NVARCHAR(8), GETDATE(), 112)
590     + N'_FULL.bak';
591 BACKUP DATABASE GradosTitulos
592     TO DISK = @ruta
593     WITH COMPRESSION, CHECKSUM,
594     DESCRIPTION = N'Backup semanal completo - GradosTitulos';
595 ',
596     @database_name = N'GradosTitulos',
597     @on_success_action = 3,
598     @on_fail_action = 2;
599 GO
600
601 EXEC sp_add_jobstep
602     @job_name = N'Mantenimiento_Semanal_GradosTitulos',
603     @step_name = N'Reorganizar_Indices',
604     @step_id = 2,
605     @subsystem = N'TSQL',
606     @command = N'
607 -- Reorganizar índices con fragmentación entre 10% y 30%
608 DECLARE @sql NVARCHAR(MAX) = N'';
609 SELECT @sql += N''ALTER INDEX [' + i.name + N']'' +
610     N'' ON [' + s.name + N'].[' + t.name + N']'' +
611     N'' REORGANIZE;' + CHAR(10)
612 FROM sys.dm_db_index_physical_stats(DB_ID(N'GradosTitulos'),NULL,NULL,NULL,'LIMITED') ps
613 JOIN sys.indexes i ON i.object_id = ps.object_id AND i.index_id = ps.index_id
614 JOIN sys.tables t ON t.object_id = i.object_id
615 JOIN sys.schemas s ON s.schema_id = t.schema_id
616 WHERE ps.avg_fragmentation_in_percent BETWEEN 10 AND 30
617     AND i.name IS NOT NULL;

```

3. Justificación Técnica

SQL Server Agent permite automatizar planes de mantenimiento mediante Jobs con múltiples pasos (steps).

El Job encadena tres tareas en orden: primero el backup (garantiza un punto de recuperación fresco antes de cualquier operación), luego la reorganización de índices (corrige fragmentación sin bloquear la tabla) y finalmente la actualización de estadísticas (mejora los planes de ejecución del optimizador de consultas). La opción `on_success_action=3` ("ir al siguiente step") encadena los pasos automáticamente. `COMPRESSION` y `CHECKSUM` en el backup reducen el tamaño del archivo y detectan corrupción al momento de escribir.

4. Buenas Prácticas Utilizadas

- Programar mantenimientos en horarios de baja actividad (madrugada / fin de semana).
- Incluir `CHECKSUM` en el backup para detectar corrupción inmediatamente, no al momento de restaurar.
- Usar `COMPRESSION` para reducir espacio en disco y tiempo de escritura/lectura del backup.
- Encadenar steps con `on_success_action=3` y redirigir errores con `on_fail_action=2` (generar alerta).
- Reorganizar (no reconstruir) índices con fragmentación entre 10-30% para evitar bloqueos prolongados.

- Separar la carpeta de backups del disco de datos para garantizar disponibilidad si el volumen de datos falla.

Proyecto 2: Política de Retención de 90 Días

1. Enunciado

Se requiere implementar una política institucional que conserve los respaldos de la base de datos

GradosTitulos durante exactamente 90 días, eliminando automáticamente los archivos más antiguos.

La política debe ejecutarse de forma automática y registrar las eliminaciones realizadas.

2. Script de Solución SQL

```

51  --5.2
52  USE msdb;
53  GO
54
55  -- =====
56  -- Job de limpieza: elimina backups con más de 90 días
57  -- =====
58  EXEC sp_add_job
59      @job_name = N'Retencion_90dias_GradosTitulos',
60      @description = N'Elimina backups físicos de más de 90 días y purga historial msdb';
61  GO
62
63  -- STEP 1: Eliminar archivos físicos de backup (> 90 días)
64  EXEC sp_add_jobstep
65      @job_name = N'Retencion_90dias_GradosTitulos',
66      @step_name = N'Eliminar_Archivos_Fisicos',
67      @step_id = 1,
68      @subsystem = N'TSQL',
69      @command = N'
70  -- sp_delete_backuphistory no elimina el archivo físico;
71  -- xp_delete_file elimina los archivos del disco.
72  EXEC xp_delete_file
73      0, -- 0 = archivo de backup
74      N'C:\SQLBackup\GradosTitulos\'',
75      N'.bak'', -- extensión
76      DATEADD(DAY, -90, GETDATE()), -- fecha límite
77      1; -- incluir subdirectorios
78  ',
79      @database_name = N'master',
80      @on_success_action = 3,
81      @on_fail_action = 2;
82  GO
83
84  -- STEP 2: Purgar historial en msdb (registros > 90 días)
85  EXEC sp_add_jobstep
86      @job_name = N'Retencion_90dias_GradosTitulos',
87      @step_name = N'Purgar_Historial_msdb',
88      @step_id = 2,
89      @subsystem = N'TSQL',
90      @command = N'
91  EXEC msdb.dbo.sp_delete_backuphistory
92      @oldest_date = DATEADD(DAY, -90, GETDATE());
93  ',
94      @database_name = N'msdb',
95      @on_success_action = 3,
96      @on_fail_action = 2;

```

3. Justificación Técnica

La política de retención combina dos mecanismos complementarios: xp_delete_file elimina los archivos físicos (.bak) del sistema de archivos, mientras que sp_delete_backuphistory limpia la tabla de historial de msdb. Ambas operaciones son

necesarias: omitir la purga de msdb provoca que la tabla backupset crezca indefinidamente, degradando el rendimiento de operaciones de restauración y consultas al catálogo. La tabla de control LogRetencion permite auditar que la política se ejecutó correctamente cada día y qué fecha de corte se aplicó.

4. Buenas Prácticas Utilizadas

- Siempre combinar xp_delete_file (disco) con sp_delete_backuphistory (msdb): omitir uno deja registros huérfanos.
- Registrar cada ejecución de purga en una tabla de control para facilitar auditorías de cumplimiento.
- Ejecutar la purga diariamente para evitar acumulación y picos de I/O si se purga semanalmente.
- Validar que la ruta en xp_delete_file incluya la barra invertida final, o la función falla silenciosamente.
- Documentar la política de retención en el plan de recuperación ante desastres (DRP) de la institución.
- Nunca reducir la retención por debajo del período exigido por regulaciones (SUNEDU puede requerir históricos).

Proyecto 3: Plan de Mantenimiento Integral

1. Enunciado

Se requiere diseñar un plan de mantenimiento integral que incluya las cuatro tareas fundamentales: backup completo, reorganización de índices, actualización de estadísticas y verificación de integridad. El plan debe ejecutarse de forma automatizada y cubrir todos los esquemas de GradosTitulos.

2. Script de Solución SQL

```

3  --S.3
4  USE msdb;
5  GO
6
7  -- =====
8  -- Job integral de mantenimiento con 4 pasos
9  -- =====
10 EXEC sp_add_job
11     @job_name = N'Mantenimiento_Integral_GradosTitulos',
12     @description = N'Backup + Indices + Estadísticas + Integridad';
13 GO
14
15 -- ---- STEP 1: Backup completo con verificación ----
16 EXEC sp_add_jobstep
17     @job_name = N'Mantenimiento_Integral_GradosTitulos',
18     @step_name = N'S1_Backup_Completo',
19     @step_id = 1,
20     @subsystem = N'TSQL',
21     @command = N'
22 DECLARE @ruta NVARCHAR(300) =
23     N''C:\SQLBackup\GradosTitulos\GT_FULL_' + CONVERT(NVARCHAR(8),GETDATE(),112) + N''.bak'';
24 BACKUP DATABASE GradosTitulos TO DISK = @ruta
25     WITH COMPRESSION, CHECKSUM, STATS = 10,
26     DESCRIPTION = N'Backup integral GradosTitulos';
27 -- Verificar el backup recién generado
28 RESTORE VERIFYONLY FROM DISK = @ruta WITH CHECKSUM;
29 '
30     @database_name = N'GradosTitulos',
31     @on_success_action = 3, @on_fail_action = 2;
32 GO
33
34 -- ---- STEP 2: Reorganización / reconstrucción de índices ----
35 EXEC sp_add_jobstep
36     @job_name = N'Mantenimiento_Integral_GradosTitulos',
37     @step_name = N'S2_Mantenimiento_Indices',
38     @step_id = 2,
39     @subsystem = N'TSQL',
40     @command = N'
41 DECLARE @sql NVARCHAR(MAX) = N''';
42 SELECT @sql +=
43     CASE
44     WHEN ps.avg_fragmentation_in_percent > 30
45     THEN N''ALTER INDEX [''+i.name+N''+'']
46         N'' ON [''+s.name+N''+''].[''+t.name+N''+'']
47         N'' REBUILD WITH (ONLINE=OFF);''+CHAR(10)
48     ELSE N''ALTER INDEX [''+i.name+N''+'']
49         N'' ON [''+s.name+N''+''].[''+t.name+N''+'']

```

3. Justificación Técnica

El plan integral sigue un orden lógico: 1) Backup antes de cualquier operación disruptiva (si REBUILD falla, se puede restaurar); 2) Mantenimiento de índices con lógica dual: REORGANIZE para fragmentación 10-30% y REBUILD para >30%, eligiendo la operación menos costosa cuando es suficiente; 3) Estadísticas con FULLSCAN en tablas críticas (Tramite, Sustentacion) para mayor precisión del optimizador; 4) CHECKDB con DATA_PURITY como verificación final detecta corrupción de páginas y valores fuera de rango. RESTORE VERIFYONLY confirma que el archivo de backup es legible antes de considerar el backup como válido.

4. Buenas Prácticas Utilizadas

- Ejecutar RESTORE VERIFYONLY inmediatamente después del backup para confirmar su integridad.
- Usar lógica condicional REORGANIZE vs REBUILD según el porcentaje de fragmentación para optimizar recursos.

- Aplicar UPDATE STATISTICS WITH FULLSCAN en tablas de alta rotación de datos (Tramite, Evaluacion).
- Ejecutar CHECKDB con DATA_PURITY para detectar también valores de columnas fuera del dominio del tipo de dato.
- Siempre poner el backup como primer step; si falla, cancelar el resto del plan.
- Usar STATS=10 en el backup para registrar el progreso en el log del Agent y facilitar el diagnóstico.

Proyecto 4: Gestión Corporativa de Retención a Cinco Años

1. Enunciado

Se requiere diseñar una política institucional de respaldo y retención que cubra cinco años de operación de la base de datos GradosTitulos. La política debe establecer diferentes niveles de granularidad (diario, semanal, mensual, anual) con distintos períodos de retención para cada uno.

2. Script de Solución SQL

```

344 --S.4
345 USE GradosTitulos;
346 GO
347
348
349
350 -- TABLA MAESTRA de política de retención
351
352 IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = N'PoliticaRetención')
353 CREATE TABLE dbo.PoliticaRetencion (
354     Id TINYINT NOT NULL IDENTITY(1,1) PRIMARY KEY,
355     TipoBackup VARCHAR(20) NOT NULL, -- FULL, DIFERENCIAL, LOG
356     Frecuencia VARCHAR(20) NOT NULL, -- DIARIO, SEMANAL, MENSUAL, ANUAL
357     DiasRetencion SMALLINT NOT NULL,
358     RutaDestino VARCHAR(300) NOT NULL,
359     Descripcion VARCHAR(200) NULL
360 );
361 GO
362
363 INSERT INTO dbo.PoliticaRetencion VALUES
364 ('FULL', 'DIARIO', 7, 'C:\SQLBackup\GradosTitulos\Diario\ ', 'Retención 1 semana'),
365 ('FULL', 'SEMANAL', 30, 'C:\SQLBackup\GradosTitulos\Semanal\ ', 'Retención 1 mes'),
366 ('FULL', 'MENSUAL', 365, 'C:\SQLBackup\GradosTitulos\Mensual\ ', 'Retención 1 año'),
367 ('FULL', 'ANUAL', 1825, 'C:\SQLBackup\GradosTitulos\Anual\ ', 'Retención 5 años'),
368 ('LOG', 'CADA_4H', 3, 'C:\SQLBackup\GradosTitulos\Logs\ ', 'Logs cada 4 horas - retención 3 días'),
369 ('DIFERENCIAL', 'DIARIO', 14, 'C:\SQLBackup\GradosTitulos\Diferencial\ ', 'Retención 2 semanas');
370 GO
371
372
373 -- SP de backup según política: recibe tipo y frecuencia
374
375 CREATE OR ALTER PROCEDURE dbo.SP_EjecutarBackupPolitica
376     @TipoBackup VARCHAR(20),
377     @Frecuencia VARCHAR(20)
378 AS
379 BEGIN
380     SET NOCOUNT ON;
381     DECLARE @ruta VARCHAR(300), @dias SMALLINT;
382
383     SELECT @ruta = RutaDestino, @dias = DiasRetencion
384     FROM dbo.PoliticaRetencion
385     WHERE TipoBackup = @TipoBackup AND Frecuencia = @Frecuencia;
386
387     DECLARE @archivo NVARCHAR(400) =
388         @ruta + 'GradosTitulos_' + @TipoBackup + '_'
389         + CONVERT(NVARCHAR(8), GETDATE(), 112)
390         + '_' + REPLACE(CONVERT(NVARCHAR(8), GETDATE(), 108), ':', '-') + '.bak';

```

3. Justificación Técnica

La estrategia de retención en niveles (GFS: Grandfather-Father-Son) es el estándar corporativo de respaldo: backups diarios (retención 7 días), semanales (1 mes), mensuales (1 año) y anuales (5 años) permiten recuperar la base de datos ante

cualquier escenario: fallo reciente (diario), error descubierto tarde (mensual) o auditoría de largo plazo (anual). La tabla `PoliticaRetencion` centraliza los parámetros de forma que un cambio en los días de retención no requiere modificar los jobs, solo actualizar la tabla maestra.

El SP `SP_EjecutarBackupPolitica` encapsula la lógica de backup y purga en un único punto de mantenimiento.

4. Buenas Prácticas Utilizadas

- Implementar la estrategia GFS (Grandfather-Father-Son) para cubrir diferentes ventanas de recuperación.
- Centralizar la política en una tabla de parámetros: los cambios de retención no requieren modificar jobs.
- Separar físicamente los backups por nivel (carpetas distintas) para aplicar políticas de retención independientes.
- Incluir la purga automática dentro del mismo SP de backup para no depender de un job adicional de limpieza.
- Conservar los backups anuales por al menos 5 años según buenas prácticas de gestión documental universitaria.
- Considerar replicar los backups anuales en almacenamiento offline o nube para protección ante desastres.

Proyecto 5: Plan Corporativo de Mantenimiento

1. Enunciado

Se requiere diseñar un plan de mantenimiento corporativo que integre de forma coordinada: respaldo completo, reorganización de índices, actualización de estadísticas y validación de integridad. El plan debe operar con umbrales configurables y enviar alertas automáticas ante errores.

2. Script de Solución SQL

```

9  --5.5
0  USE msdb;
1  GO
2
3  -- =====
4  -- Operador de alertas (recibe notificaciones de error)
5  -- =====
6  IF NOT EXISTS (SELECT 1 FROM msdb.dbo.sysoperators WHERE name = N'DBA_GradosTitulos')
7      EXEC sp_add_operator
8          @name = N'DBA_GradosTitulos',
9          @email_address = N'dba@upla.edu.pe';
0  GO
1
2  -- =====
3  -- Job corporativo con notificación al operador en caso de falla
4  -- =====
5  EXEC sp_add_job
6      @job_name = N'Plan_Corporativo_Mant_GT',
7      @description = N'Plan corporativo: Backup+Indices+Stats+Integridad con alertas',
8      @notify_level_eventlog = 2, -- log si falla
9      @notify_level_email = 2, -- email si falla
0      @notify_email_operator_name = N'DBA_GradosTitulos';
1  GO
2
3  -- ---- STEP 1: Backup completo ----
4  EXEC sp_add_jobstep
5      @job_name = N'Plan_Corporativo_Mant_GT',
6      @step_name = N'Backup_Completo', @step_id = 1,
7      @subsystem = N'TSQL',
8      @command = N'
9  DECLARE @ruta NVARCHAR(300) =
0      N'C:\SQLBackup\GradosTitulos\GT_CORP_' + CONVERT(NVARCHAR(8),GETDATE(),112) + N''.bak'';
1  BACKUP DATABASE GradosTitulos TO DISK = @ruta
2      WITH COMPRESSION, CHECKSUM, STATS = 10;
3  RESTORE VERIFYONLY FROM DISK = @ruta WITH CHECKSUM;
4  '
5      , @database_name = N'GradosTitulos',
6      @on_success_action = 3, @on_fail_action = 2;
7  GO
8
9  -- ---- STEP 2: Reorganización / reconstrucción de índices ----
0  EXEC sp_add_jobstep
1      @job_name = N'Plan_Corporativo_Mant_GT',
2      @step_name = N'Mantenimiento_Indices', @step_id = 2,
3      @subsystem = N'TSQL',
4      @command = N'
5  DECLARE @sql NVARCHAR(MAX) = N'';
6  SELECT @sql += CASE

```

3. Justificación Técnica

El plan corporativo agrega sobre el plan integral el componente de notificaciones automáticas: el operador DBA_GradosTitulos recibe un correo electrónico cuando cualquier step falla, eliminando la dependencia de revisiones manuales. El parámetro notify_level_email=2 envía alerta solo en caso de fallo (no en éxito), evitando saturar el correo. La combinación de notify_level_eventlog=2 asegura que el fallo queda también en el Event Log de Windows, permitiendo correlación con herramientas de monitoreo externas como Nagios o Azure Monitor. FULLSCAN en tablas de documentos oficiales (Diploma) garantiza estadísticas precisas.

4. Buenas Prácticas Utilizadas

- Configurar siempre un operador de alertas en msdb para recibir notificaciones automáticas de fallo.
- Usar notify_level_email=2 (solo en fallo) para no generar ruido innecesario en el correo del DBA.

- Combinar notify_level_eventlog con notify_level_email para trazabilidad en múltiples sistemas.
- Aplicar FULLSCAN en tablas con datos críticos y baja volatilidad para máxima precisión del optimizador.
- Documentar el plan en el DRP incluyendo el contacto del operador y el procedimiento manual alternativo.
- Revisar periódicamente el historial del Job en msdb.dbo.sysjobhistory para detectar tendencias de degradación.

Proyecto 6: Política de Retención Automatizada de 180 Días

1. Enunciado

Se requiere diseñar una política institucional que elimine automáticamente los respaldos con más de 180 días de antigüedad. La política debe aplicarse a todos los tipos de backup (FULL, diferencial y logs de transacciones) y registrar un historial de las eliminaciones realizadas.

2. Script de Solución SQL

```
--5.6
USE GradosTitulos;
GO

-- =====
-- Tabla de auditoría de purgas
-- =====
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = N'AuditoriaPurga')
CREATE TABLE dbo.AuditoriaPurga (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    FechaEjecucion DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    TipoBackup VARCHAR(20) NOT NULL,
    RutaPurgada NVARCHAR(300) NOT NULL,
    FechaCorte DATE NOT NULL,
    DiasRetencion SMALLINT NOT NULL,
    Resultado VARCHAR(20) NOT NULL -- EXITOSO, ERROR
);
GO

-- =====
-- SP de purga configurable por tipo de backup
-- =====
CREATE OR ALTER PROCEDURE dbo.SP_PurgarBackups
    @TipoBackup VARCHAR(20),
    @RutaBackup NVARCHAR(300),
    @DiasRetencion SMALLINT = 180
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @FechaCorte DATETIME = DATEADD(DAY, -@DiasRetencion, GETDATE());
    DECLARE @extension NVARCHAR(10) =
        CASE @TipoBackup WHEN N'LOG' THEN N'trn' ELSE N'bak' END;

    BEGIN TRY
        EXEC xp_delete_file 0, @RutaBackup, @extension, @FechaCorte, 1;

        -- También purgar historial en msdb
        EXEC msdb.dbo.sp_delete_backuphistory @oldest_date = @FechaCorte;

        INSERT INTO dbo.AuditoriaPurga (TipoBackup, RutaPurgada, FechaCorte, DiasRetencion, Resultado)
        VALUES (@TipoBackup, @RutaBackup, CAST(@FechaCorte AS DATE), @DiasRetencion, N'EXITOSO');
    END TRY
    BEGIN CATCH
        INSERT INTO dbo.AuditoriaPurga (TipoBackup, RutaPurgada, FechaCorte, DiasRetencion, Resultado)
        VALUES (@TipoBackup, @RutaBackup, CAST(@FechaCorte AS DATE), @DiasRetencion,
            N'ERROR: ' + ERROR_MESSAGE());
    END CATCH;
END
```

3. Justificación Técnica

La política de 180 días aplica a los tres tipos de backup con extensiones distintas (.bak para FULL/diferencial. trn para logs de transacciones). El SP SP_PurgarBackups es parametrizable, de modo que modificar los días de retención solo requiere cambiar el parámetro en el Job, sin reescribir código. El bloque TRY/CATCH garantiza que un error en la purga de un tipo de backup no cancele la purga de los demás (los steps usan on_fail_action=3 para continuar al siguiente). La tabla AuditoriaPurga proporciona evidencia de cumplimiento de la política para auditorías institucionales o regulatorias externas.

4. Buenas Prácticas Utilizadas

- Hacer el SP parametrizable: los días de retención deben configurarse sin modificar el código.
- Usar TRY/CATCH en la purga y registrar errores en la tabla de auditoría, no solo ignorarlos.
- Distinguir la extensión correcta por tipo (.bak vs .trn) para no eliminar archivos equivocados.
- Aplicar on_fail_action=3 (ir al siguiente step) en purgas independientes para no cancelar la cadena.
- Mantener un historial de purgas en una tabla de auditoría para evidencia de cumplimiento normativo.
- Revisar AuditoriaPurga regularmente para detectar purgas fallidas que acumularán espacio en disco.

TEMA 6 — Restauración Punto en el Tiempo (Point-in-Time Recovery)

REQUISITO PREVIO: Para restauración punto en el tiempo, la base de datos debe estar en modelo de recuperación FULL o BULK_LOGGED. Ejecutar una sola vez si aún no está configurado:

Proyecto 1: Recuperación Antes de una Eliminación Accidental

► 1. Enunciado

Un operador eliminó registros de expedientes de la tabla Tramite.Tramite a las 10:15 a.m. Se requiere restaurar la base de datos exactamente a las 10:14 a.m., recuperando los registros

eliminados sin perder las transacciones anteriores a ese momento.

► 2. Script de Solución SQL

```
1132 | -- Verificar modelo de recuperación actual
1133 | SELECT name, recovery_model_desc FROM sys.databases WHERE name = N'GradosTitulos';
1134 | -- Cambiar a FULL si es necesario
1135 | ALTER DATABASE GradosTitulos SET RECOVERY FULL;
1136 | -- Tomar un backup completo inicial (requerido para activar el log chain)
1137 | BACKUP DATABASE GradosTitulos TO DISK = N'C:\SQLBackup\GradosTitulos\GT_FULL_INICIAL.bak'
1138 | WITH COMPRESSION, CHECKSUM;
1139 |
```

► 3. Justificación Técnica

La restauración punto en el tiempo requiere la cadena log: backup completo → backup(s) de log → tail-log backup.

El tail-log backup (PASO 2) es esencial: captura las transacciones del log activo que aún no tienen un backup

formal. Sin él, se pierden todas las transacciones ocurridas desde el último backup de log programado.

NORECOVERY mantiene la BD en estado "Restoring" entre pasos, permitiendo aplicar más archivos de log.

STOPAT especifica el momento exacto de parada: 10:14:59 coloca la BD exactamente antes del DELETE de las 10:15.

REPLACE es necesario cuando se restaura sobre una BD existente con diferente GUID de backup.

✓ 4. Buenas Prácticas Utilizadas

- Siempre tomar el tail-log backup antes de iniciar la restauración para no perder transacciones recientes.
- Documentar la hora exacta del incidente antes de comenzar: STOPAT debe ser preciso al segundo.
- Usar NORECOVERY en todos los pasos intermedios y RECOVERY solo en el último paso.
- Verificar la cadena de backups en msdb antes de comenzar para evitar "broken log chain" durante la restauración.
- Restaurar primero en un servidor de prueba (si hay tiempo) para confirmar el punto de recuperación correcto.
- Tras la restauración, ejecutar DBCC CHECKDB para confirmar la integridad de la base restaurada.

Proyecto 2: Recuperación de Cambios Incorrectos en Estados de Trámites

► 1. Enunciado

Un proceso masivo actualizó incorrectamente los estados de trámites en la tabla Tramite.Tramite, modificando registros que no debían ser alterados. Se requiere restaurar la base de datos a un punto anterior al proceso incorrecto, preservando todas las demás transacciones válidas.

► 2. Script de Solución SQL

```

1140 --6.2
1141 -- =====
1142 -- PASO 1: Identificar los backups disponibles para la cadena
1143 -- =====
1144 USE msdb;
1145 GO
1146
1147 SELECT bs.backup_set_id, bs.type AS TipoBackup,
1148        bs.backup_start_date, bs.backup_finish_date,
1149        bmf.physical_device_name AS Archivo
1150 FROM   backupset bs
1151 JOIN   backupmediafamily bmf ON bmf.media_set_id = bs.media_set_id
1152 WHERE  bs.database_name = N'GradosTitulos'
1153        AND bs.backup_start_date <= CAST(GETDATE() AS DATE) -- backups de hoy
1154 ORDER BY bs.backup_set_id DESC;
1155 GO
1156
1157 -- =====
1158 -- PASO 2: Backup del log de transacciones ACTIVO (cola del log)
1159 -- CRÍTICO: captura transacciones ocurridas hasta ahora
1160 -- =====
1161 BACKUP LOG GradosTitulos
1162 TO DISK = N'C:\SQLBackup\GradosTitulos\Logs\GT_LOG_tail_10h15.trn'
1163 WITH NORECOVERY, CHECKSUM,
1164     DESCRIPTION = N'Tail-log backup antes de restauración punto en tiempo';
1165 GO
1166
1167 -- =====
1168 -- PASO 3: Restaurar el backup completo en modo NORECOVERY
1169 -- =====
1170 RESTORE DATABASE GradosTitulos
1171 FROM DISK = N'C:\SQLBackup\GradosTitulos\GT_FULL_<fecha>.bak'
1172 WITH NORECOVERY,
1173     REPLACE, -- sobrescribir la BD actual
1174     STATS = 10;
1175 GO
1176
1177 -- =====
1178 -- PASO 4: Aplicar backups de log hasta las 10:14:59
1179 -- =====
1180 RESTORE LOG GradosTitulos
1181 FROM DISK = N'C:\SQLBackup\GradosTitulos\Logs\GT_LOG_tail_10h15.trn'
1182 WITH NORECOVERY,
1183     STOPAT = N'2024-06-10 10:14:59'; -- 1 segundo ANTES del DELETE
1184 GO

```

► 3. Justificación Técnica

Antes de iniciar la restauración, el uso de la auditoría SQL Server (configurada en el Tema anterior) permite identificar la hora exacta del UPDATE masivo con precisión de segundos, lo que hace el STOPAT más confiable.

Si no hay auditoría, se puede consultar sys.fn_dblog (log de transacciones activo) o los registros de la aplicación. La query de verificación al final confirma la distribución de estados en la tabla Tramite,

lo que permite comparar contra el estado esperado antes del proceso incorrecto.

✓ 4. Buenas Prácticas Utilizadas

- Usar la auditoría SQL Server para localizar la hora exacta del incidente con precisión de segundos.
- Si no hay auditoría, consultar sys.fn_dblog antes del tail-log backup para no perder el log activo.
- Documentar el estado esperado de los datos ANTES de restaurar para poder verificar la recuperación.
- Considerar restaurar en paralelo en un servidor alternativo para reducir el tiempo de inactividad.

- Tras la recuperación, revisar el proceso que generó el UPDATE masivo y agregar controles de validación.
- Implementar una transacción con BEGIN TRAN y verificación previa en procesos masivos de actualización.

Proyecto 3: Recuperación Selectiva de Incidente Crítico (Tramite y Evaluacion)

► 1. Enunciado

Un error en producción afectó simultáneamente las tablas de los esquemas Tramite y Evaluacion.

Se requiere diseñar una recuperación punto en el tiempo minimizando la indisponibilidad del sistema,

usando una base de datos auxiliar para mantener el servicio mientras se restaura la principal.

► 2. Script de Solución SQL

```
--6.3
-- =====
-- PASO 1: Identificar la hora exacta del UPDATE masivo incorrecto
-- (consultar logs de aplicación o auditoría SQL Server)
-- =====
USE GradosTitulos;
GO

-- Si hay auditoría activa, buscar el evento:
SELECT TOP 20 event_time, server_principal_name, statement
FROM sys.fn_get_audit_file('C:\SQLAudit\GradosTitulos\DML\*.sqlaudit',DEFAULT,DEFAULT)
WHERE schema_name = 'Tramite' AND action_id = 'UP'
ORDER BY event_time DESC;
GO

-- =====
-- PASO 2: Tail-log backup del log activo
-- =====
BACKUP LOG GradosTitulos
TO DISK = N'C:\SQLBackup\GradosTitulos\Logs\GT_LOG_tail_prerestauracion.trn'
WITH NORECOVERY, CHECKSUM;
GO

-- =====
-- PASO 3: Restaurar el backup completo base
-- =====
RESTORE DATABASE GradosTitulos
FROM DISK = N'C:\SQLBackup\GradosTitulos\GT_FULL_<fecha>.bak'
WITH NORECOVERY, REPLACE, STATS = 10;
GO

-- =====
-- PASO 4: Aplicar logs hasta el momento anterior al UPDATE
-- (suponer que el proceso incorrecto se ejecutó a las 14:30)
-- =====
-- Si hay logs intermedios, aplicarlos primero en NORECOVERY:
-- RESTORE LOG GradosTitulos FROM DISK = N'...' WITH NORECOVERY;

RESTORE LOG GradosTitulos
FROM DISK = N'C:\SQLBackup\GradosTitulos\Logs\GT_LOG_tail_prerestauracion.trn'
WITH NORECOVERY,
STOPAT = N'2024-06-10 14:29:59'; -- 1 segundo antes del proceso masivo
GO

-- =====
-- PASO 5: Recuperar la BD
-- =====
RESTORE DATABASE GradosTitulos WITH RECOVERY;
```

► 3. Justificación Técnica

La recuperación selectiva evita el downtime total: mientras la BD principal permanece en línea (con datos corruptos), se restaura una BD auxiliar (GradosTitulos_Recovery) con la cláusula MOVE para redirigir archivos.

Los datos correctos se exportan con SELECT INTO a tablas de staging dentro de la BD principal, y luego

se reemplazan en una transacción atómica. Este enfoque minimiza la ventana de indisponibilidad a los minutos

que tarda el DELETE+INSERT de las tablas afectadas, en lugar de la restauración completa de toda la BD.

✓ 4. Buenas Prácticas Utilizadas

- Usar la cláusula MOVE en RESTORE para redirigir archivos y no interferir con la BD de producción.
- Restaurar en una BD auxiliar para mantener el servicio disponible (aunque con datos degradados) durante la recuperación.
- Usar SELECT INTO para exportar datos rápidamente entre bases de datos del mismo servidor.
- Encapsular el reemplazo de datos en una transacción explícita para garantizar atomicidad.
- Deshabilitar constraints durante la reimportación y re-habilitarlos antes del COMMIT para evitar errores de FK.
- Eliminar la BD auxiliar y las tablas de staging inmediatamente después de confirmar la recuperación exitosa.

Proyecto 4: Simulación de Recuperación Forense Institucional

► 1. Enunciado

Se detectó una modificación maliciosa realizada durante una ventana específica de tiempo en la base de datos GradosTitulos. Se requiere una recuperación forense que restaure la base exactamente al momento anterior al incidente y documente todo el procedimiento de forma trazable.

► 2. Script de Solución SQL

```

260 --6.4
261 --6.4
262 --=====
263 -- FASE FORENSE 1: Documentar el incidente ANTES de actuar
264 --=====
265 USE GradosTitulos;
266 GO
267
268 -- 1a. Capturar snapshot del estado actual (evidencia del daño)
269 SELECT SYSDATETIME() AS MomentoCaptura,
270        t.IdTramite, t.IdEstadoTramite, t.FechaModificacion
271 INTO    dbo.Evidencia_Incidente_20240610
272 FROM    Tramite.Tramite t;
273
274 SELECT SYSDATETIME() AS MomentoCaptura, d.*
275 INTO    dbo.Evidencia_Documento_20240610
276 FROM    Documento.Diploma d;
277 GO
278
279 -- 1b. Consultar auditoría para identificar la ventana del incidente
280 SELECT event_time, server_principal_name, client_ip,
281        schema_name, object_name, action_id, statement
282 FROM sys.fn_get_audit_file('C:\SQLAudit\GradosTitulos\Institucional\*.sqlaudit', DEFAULT, DEFAULT)
283 WHERE event_time BETWEEN '2024-06-10 08:00' AND '2024-06-10 10:00'
284        AND action_id IN ('IN', 'UP', 'DL')
285 ORDER BY event_time;
286 GO
287
288 --=====
289 -- FASE FORENSE 2: Tail-log backup (preservar evidencia del log)
290 --=====
291 USE master; GO
292 BACKUP LOG GradosTitulos
293     TO DISK = N'C:\SQLBackup\Forenses\GT_LOG_FORENSE_tail.trn'
294     WITH NORECOVERY, CHECKSUM,
295     DESCRIPTION = N'Tail-log forense - evidencia de incidente 2024-06-10';
296 GO
297
298 --=====
299 -- FASE FORENSE 3: Restaurar a punto anterior al incidente
300 -- (suponer que el incidente inició a las 08:15)
301 --=====
302 RESTORE DATABASE GradosTitulos
303     FROM DISK = N'C:\SQLBackup\GradosTitulos\GT_FULL_20240610.bak'
304     WITH NORECOVERY, REPLACE, STATS = 10;
305 GO

```

► 3. Justificación Técnica

La recuperación forense añade una fase de documentación pre-restauración crítica: las tablas de evidencia (Evidencia_Incidente_20240610) preservan el estado del daño como prueba, y el tail-log backup en una carpeta separada (Forense) garantiza que no se sobrescriba con los logs programados. La auditoría institucional (configurada en el Tema anterior) permite identificar con precisión la ventana de tiempo del ataque. El InformeForense documenta el procedimiento de recuperación, creando trazabilidad para reportes a autoridades universitarias o investigaciones de seguridad.

✓ 4. Buenas Prácticas Utilizadas

- Documentar el estado del daño (tablas de evidencia) ANTES de iniciar cualquier restauración.
- Guardar el tail-log forense en una carpeta separada para preservarlo como evidencia.
- Usar la auditoría SQL Server para delimitar la ventana temporal del ataque con precisión de segundos.
- Crear un InformeForense con fecha, responsable y acciones tomadas para trazabilidad institucional.
- No eliminar las tablas de evidencia hasta que la investigación interna o legal esté cerrada.

- Cambiar todas las contraseñas y revisar los permisos inmediatamente después de la recuperación forense.

Proyecto 5: Recuperación Exacta de Expedientes Eliminados a las 11:44:59

► 1. Enunciado

A las 11:45 a.m. se eliminaron accidentalmente registros de expedientes de titulación de la base de datos GradosTitulos. Se requiere recuperar la base exactamente al estado que tenía

a las 11:44:59 a.m., garantizando la recuperación completa de los expedientes eliminados.

► 2. Script de Solución SQL

```

352 --6.5
353 -- =====
354 -- PASO 1: Confirmar el incidente consultando auditoría
355 -- =====
356 USE GradosTitulos; GO
357
358 SELECT event_time, server_principal_name, client_ip, statement
359 FROM sys.fn_get_audit_file('C:\SQLAudit\GradosTitulos\DML\*.sqlaudit', DEFAULT, DEFAULT)
360 WHERE action_id = 'DL'
361       AND schema_name IN ('Tramite', 'Academico')
362       AND event_time BETWEEN '2024-06-10 11:40' AND '2024-06-10 11:50'
363 ORDER BY event_time;
364 GO
365
366 -- =====
367 -- PASO 2: Tail-log backup
368 -- =====
369 USE master; GO
370 BACKUP LOG GradosTitulos
371 TO DISK = N'C:\SQLBackup\GradosTitulos\Logs\GT_LOG_tail_11h45.trn'
372 WITH NORECOVERY, CHECKSUM,
373     DESCRIPTION = N'Tail-log - recuperación expedientes eliminados 11:45';
374 GO
375
376 -- =====
377 -- PASO 3: Restaurar backup completo en NORECOVERY
378 -- =====
379 RESTORE DATABASE GradosTitulos
380 FROM DISK = N'C:\SQLBackup\GradosTitulos\GT_FULL_<hoy>.bak'
381 WITH NORECOVERY, REPLACE, STATS = 10;
382 GO
383
384 -- =====
385 -- PASO 4: Aplicar logs intermedios (si los hay) en NORECOVERY
386 -- =====
387 -- Si existen backups de log entre el FULL y las 11:44, aplicarlos:
388 RESTORE LOG GradosTitulos
389 FROM DISK = N'C:\SQLBackup\GradosTitulos\Logs\GT_LOG_09h00.trn'
390 WITH NORECOVERY;
391 GO
392
393 -- =====
394 -- PASO 5: Aplicar tail-log con STOPAT exacto
395 -- =====
396 RESTORE LOG GradosTitulos
397 FROM DISK = N'C:\SQLBackup\GradosTitulos\Logs\GT_LOG_tail_11h45.trn'
398 WITH NORECOVERY,
399     STOPAT = N'2024-06-10 11:44:59'; -- exactamente 1 segundo antes del DELETE

```

► 3. Justificación Técnica

Este ejercicio refleja un escenario idéntico al Proyecto 1 del mismo tema, pero aplicado específicamente a

expedientes de titulación. El punto clave técnico es el STOPAT = 11:44:59, que detiene la aplicación del log un segundo antes del DELETE accidental. SQL Server garantiza que todas las transacciones COMMIT antes de ese timestamp quedan aplicadas y las posteriores son descartadas. La consulta de verificación usa los tipos de trámite (2, 3, 4 = titulación) para confirmar que los expedientes fueron recuperados correctamente.

✓ 4. Buenas Prácticas Utilizadas

- Confirmar la hora exacta del DELETE en la auditoría ANTES de definir el STOPAT.
- STOPAT debe ser siempre un segundo antes del DELETE: 11:44:59, no 11:45:00.
- Aplicar todos los logs intermedios entre el FULL y el tail-log para no romper la cadena de recuperación.
- Verificar la integridad con DBCC CHECKDB inmediatamente después de cada restauración en producción.
- Documentar el número de registros recuperados comparando contra registros en el sistema de expedientes.
- Configurar backups de log cada 15-30 minutos en bases de datos críticas para reducir la ventana de pérdida.

Proyecto 6: Recuperación Forense de Modificación Maliciosa en Documento

► 1. Enunciado

Se detectó una modificación no autorizada en las tablas del esquema Documento (Resolución y Diploma).

Se requiere una recuperación punto en el tiempo que restaure el estado exacto de la base de datos

antes de la alteración, preservando evidencias para el proceso de investigación institucional.

► 2. Script de Solución SQL


```

23 |
24 | --6.6
25 |
26 | =====
27 | -- FASE 1: PRESERVAR EVIDENCIAS (ejecutar INMEDIATAMENTE)
28 | =====
29 | USE GradosTitulos; GO
30 |
31 | -- Capturar estado actual de los documentos comprometidos
32 | SELECT SYSDATETIME() AS FechaEvidencia, r.*
33 | INTO    dbo.Evidencia_Resolucion_Comprometida
34 | FROM    Documento.Resolucion r;
35 |
36 | SELECT SYSDATETIME() AS FechaEvidencia, d.*
37 | INTO    dbo.Evidencia_Diploma_Comprometido
38 | FROM    Documento.Diploma d;
39 | GO
40 |
41 | -- Consultar auditoría: ¿quién y cuándo modificó el esquema Documento?
42 | SELECT event_time, server_principal_name, client_ip,
43 |        object_name AS Tabla, action_id AS Operacion, statement
44 | FROM sys.fn_get_audit_file('C:\SQLAudit\GradosTitulos\Institucional\*.sqlaudit', DEFAULT, DEFAULT)
45 | WHERE schema_name = 'Documento'
46 |        AND action_id IN ('IN', 'UP', 'DL')
47 |        AND event_time >= DATEADD(DAY, -7, SYSDATETIME())
48 | ORDER BY event_time DESC;
49 | GO
50 |
51 | =====
52 | -- FASE 2: Tail-log backup forense (preservar log como evidencia)
53 | =====
54 | USE master; GO
55 |
56 | BACKUP LOG GradosTitulos
57 | TO DISK = N'C:\SQLBackup\Forense\GT_LOG_FORENSE_Documento.trn'
58 | WITH NORECOVERY, CHECKSUM,
59 |     DESCRIPTION = N'LOG FORENSE - Modificación no autorizada esquema Documento';
60 | GO
61 |
62 | =====
63 | -- FASE 3: Restaurar a un punto anterior a la modificación
64 | -- (suponer que la auditoria indica que el ataque fue a las 03:22)
65 | =====
66 | RESTORE DATABASE GradosTitulos
67 | FROM DISK = N'C:\SQLBackup\GradosTitulos\GT_FULL_<fecha>.bak'
68 | WITH NORECOVERY, REPLACE, STATS = 10;
69 | GO

```

► 3. Justificación Técnica

El esquema Documento contiene los registros más críticos de la institución (resoluciones y diplomas oficiales).

La preservación de evidencias en tablas de staging antes de iniciar la restauración es un requisito forense

fundamental: permite comparar el estado comprometido con el estado recuperado, y sirve como evidencia en

investigaciones disciplinarias o legales. La combinación de auditoría institucional (Tema anterior) y el

tail-log forense permite delimitar la ventana del ataque con precisión de segundos. El STOPAT se posiciona

con un margen de 1 minuto antes del ataque para garantizar que no se incluya ninguna transacción maliciosa.

✓ 4. Buenas Prácticas Utilizadas

- Preservar evidencias del estado comprometido en tablas de staging ANTES de restaurar (requisito forense).
- Guardar el tail-log forense en una carpeta separada para preservarlo como evidencia del incidente.
- Usar la auditoría institucional para identificar con precisión la ventana del ataque.

- Crear un InformeForense completo que incluya fecha del incidente, STOPAT utilizado y responsable.
- No eliminar las tablas de evidencia hasta que la investigación institucional o legal esté cerrada formalmente.
- Tras la recuperación, revisar y reforzar inmediatamente el control de acceso al esquema Documento.